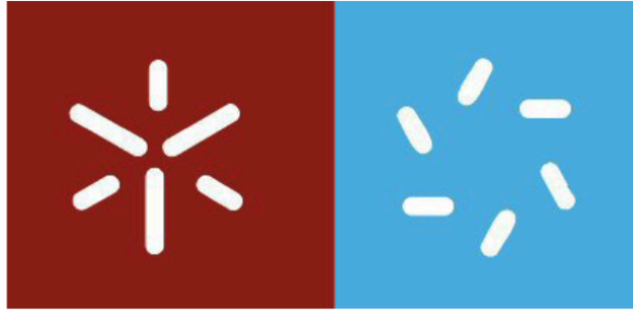




Universidade do Minho
Licenciatura em Ciências da Computação

Sistemas Operativos - Trabalho Prático

João Pontes
(A111657)

João Cascais
(A110393)

Índice

1. Introdução.....	3
2. Contexto e Funcionalidades.....	4
2.1. MakeFile.....	4
2.2. lib.h.....	4
2.3. Funcionalidades Básicas.....	4
2.3.1. "\$ -exit".....	4
2.3.2. "\$ -status".....	5
2.3.3. Scheduling Policies.....	5
2.3.4. Tempo de execução.....	5
2.4. Funcionalidades Avançadas.....	5
2.4.1. Comandos em PipeLine e em Paralelo.....	5
3. Implementações.....	6
3.1. Lógica de Implementação.....	6
Figura 1 - Lógica do Programa Runner-Controller.....	6
3.2. Cliente (Runner.c).....	7
3.2.1. .main().....	7
3.2.2. .dispatch().....	8
3.2.3. .parser().....	8
3.2.4. .cutpipe().....	8
3.2.5. .executor().....	8
3.3. Servidor (Controller.c).....	8
3.3.1. .main().....	8
3.3.2. .process().....	9
3.3.3. .manager().....	9
3.3.4. .enqueue().....	9
3.3.5. .dequeue().....	10
4. Considerações Finais.....	10

1. Introdução

Foi-nos proposto, no âmbito da UC Sistemas Operativos, a criação de uma comunicação em contexto Cliente - Servidor. Mais concretamente, um serviço de comandos(tasks/commands) , em que o cliente(Runner) envia, primeiramente um pedido de execução ao servidor(Controller), e após feedback do mesmo, o cliente executa ou não o/(s) comando/(s). Notoriamente, o servidor é capaz de gerir os pedidos, através de uma *Queue* juntamente com uma, ou até mesmo várias, *scheduling policies* (políticas de escalonamento).

O cliente e o servidor comunicam através de dois FIFO's (First In First Out), mais concretamente, *pipelines* com nome. A primeira “**temp/Communicator**”, sendo capaz de receber vários pedidos vindos do/(s) cliente/(s) para o servidor, e a segunda “**temp/comm_**” sendo um FIFO temporário para um dado cliente. Há que notar que sempre que um cliente executa um comando, notifica o servidor do sucesso/ insucesso da execução (juntamente com o tempo de execução do mesmo) e logo de seguida, fecha juntamente com o seu FIFO correspondente.

2. Contexto e Funcionalidades

2.1. MakeFile

No início da execução do programa, deverá ser respeitada a composição do MakeFile do projeto. Para compilar o programa deve ser digitado em qualquer terminal, “**\$ make**”. Para iniciar o servidor, deverá ser digitado o seguinte comando: “**\$./bin/Controller**”, e para iniciar um cliente: “**\$./bin/Runner <commands> (i)**”. Por fim, para limpar a diretoria de ficheiros temporários (“**temp**”), basta digitar em qualquer terminal “**\$ make clean**”.

(i) - a referência “**<commands>**” é dada pela lista de comandos a executar (ex: “**\$ ls | cat Runner.c**”).

2.2. lib.h

No ficheiro “lib.h” reside a estrutura base do programa, “Programa”, bem como a definição da *Queue*. Está também definido no ficheiro o máximo de comandos a executar em simultâneo (**MAX_PARALLEL**) assim como o tamanho máximo da *Queue* (**MAX_QUEUE**).

Note-se que: “**define MAX_PARALLEL 2**” & “**define MAX_QUEUE 100**”.

2.3. Funcionalidades Básicas

2.3.1. “\$ -exit”

Em (i) foi referido que **<commands>** representava a lista de comandos a executar. Ora se essa lista for composta por um conjunto de comandos/funcionalidades específicas como o “**\$ -exit**”, o programa tem respostas/comportamentos pré-definidos. Ao digitar no terminal do cliente “**\$./bin/Runner -exit**”, o servidor encerra para pedidos. Este comando serve para fechar os canais de comunicação entre um dado Cliente e um Servidor. Ou seja, o Servidor deixa, automaticamente, de receber novos pedidos.

Note-se que: se o último comando digitado for, por exemplo: “\$./bin/Runner ls & ./bin/Runner -exit & ./bin/Runner ping -c 5 uminho.pt”, o último elemento da lista de comandos “\$ ping -c 5 uminho.pt” não vai, obviamente, ser executado.

2.3.2. “\$ -status”

Parte do conjunto de comandos/funcionalidades específicas, faz parte também, o comando “\$ -status”. O cliente ao digitar “\$./bin/Runner -status”, pode averiguar sobre os comandos em execução (ou lista de espera).

2.3.3. *Scheduling Policies*

No contexto do programa foi utilizada como política de escalonamento base, a **FCFS** (First-Come, First-Served). Os comandos são executados pela ordem de chegada, independentemente da urgência, tamanho ou utilidade dos mesmos.

2.3.4. Tempo de execução

O tempo de execução é obtido através da função da biblioteca em C “<sys/time.h>”, `.gettimeofday()`.

2.4. Funcionalidades Avançadas

2.4.1. Comandos em *PipeLine* e em Paralelo

O programa suporta que um utilizador digite uma série de comandos em *pipeline* (ex: “\$./bin/Runner sleep 10 | ls | cat lib.h”, ou “\$./bin/Runner ls | grep .txt > list.txt”).

O Programa também suporta a execução de dois (ou mais dependendo da definição) comandos paralelos, de um ou vários utilizadores, visto que “**define MAX_PARALLEL 2**” (ex: Cliente 1: “\$./bin/Runner ls -la & ./bin/Runner sleep 5”; Cliente 2: \$./bin/Runner ls | wc).

3. Implementações

3.1. Lógica de Implementação

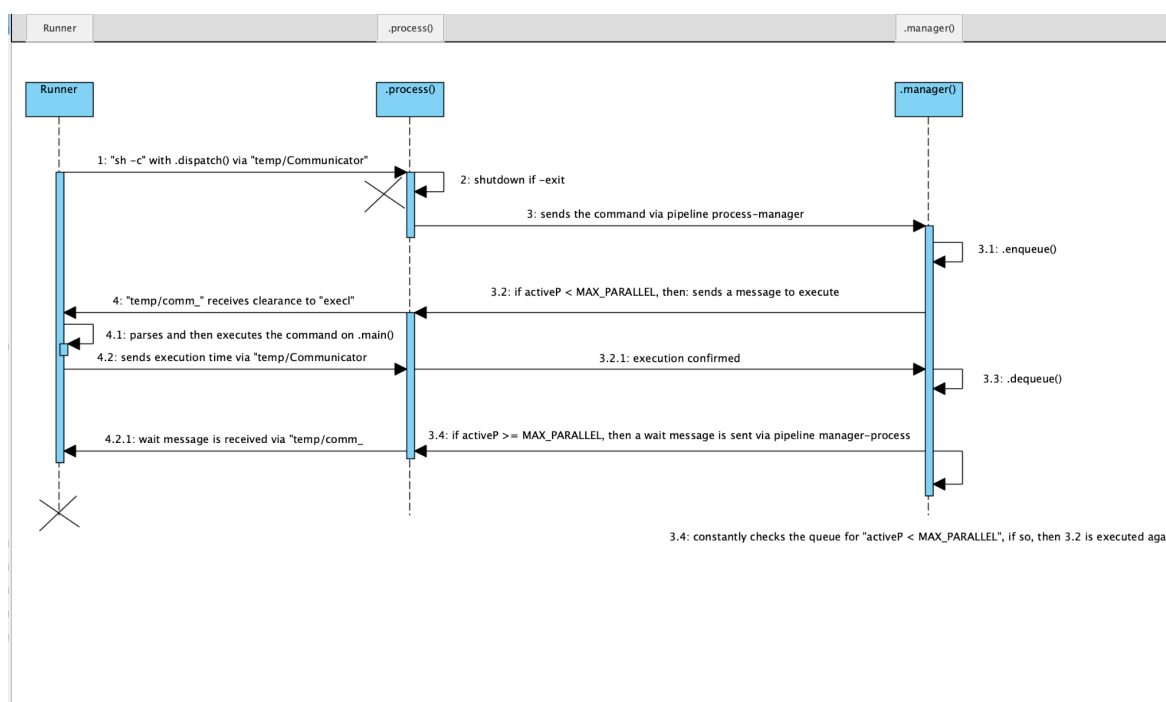


Figura 1 - Lógica do Programa Runner-Controller

A figura acima ilustrada, representa, de modo bastante superficial, o funcionamento da comunicação Cliente - Servidor.

Resumidamente, o programa é composto por dois principais ficheiros ".c". O Controller, que trata de receber pedidos de execução de comandos de um ou mais Clientes, dando ou não permissão para os executar. A permissão é dada a partir do momento em que os processos ativos são menores que o máximo de processos a executar em paralelo (**activeP < MAX_PARALLEL**). O Runner, responsabiliza-se pelo envio de pedidos ao Controller, se receber permissão de execução, executa os comandos, enviando o tempo de execução dos mesmos ao Controller. Se a permissão de execução for posta em *standby* (*Queued process*), o Runner espera poder executar a lista de comandos.

A comunicação (indireta) entre a fila de espera e o Runner é feita por duas funções auxiliares: Manager (**.manager()**) e Process (**.process()**). A função Process, trata de receber pedidos via “**temp/Communicator**” (pipeline com nome do Controller), enviando-os via pipeline1 para o Manager. O Manager recebe esses comandos, e através de outras duas funções auxiliares: *Enqueue* (**.enqueue()**) e *Dequeue* (**.dequeue()**). Funções essas que são capazes de gerir a fila de espera (*Queue*). Seguidamente o Manager envia o comando ou resposta(resposta de espera) via pipeline2 para o Process, sendo o mesmo encarregue de comunicar ao Cliente o estado do seu pedido.

3.2. Cliente (**Runner.c**)

3.2.1. **.main()**

A função **.main()** responsabiliza-se pela construção do comando a ser enviado para o Controller. O comando é enviado, primeiramente em forma de pedido, só após receber permissão de execução, é executado. São feitas verificações iniciais ao input do Cliente.

- **if(argc < 2)**

Se o conjunto de comandos for menor que 2, i.e. se o input do utilizador for: “**\$./bin/Runner**” o próprio Runner alerta para a falta de argumentos na expressão: “**\$ Usage: %s <commands> -> missing args**”

- **if(strcmp(argv[i], “(*)”) == 0)**

Se o primeiro índice da lista de comandos pertencer à lista de funcionalidades especiais do programa, o Runner recebe imediatamente resposta do Controller.

Note-se que: (*) -> “**\$ -exit**” ou “**\$ -status**”.

- **if(dispatch(argc, argv) == -1)**

Se o retorno da função **.dispatch()** for -1, o programa avisa que houve erro, ou no envio do comando, ou na resposta do Controller. Senão, avança para a execução do comando.

Após estas verificações iniciais, o restante da função é dado, primeiramente pela construção do comando com **char actual_command[256]** seguido de um *for loop* com **.strncat()** (redirecionando o input do Runner, para um buffer.). É depois, criado um **pid = fork()**, onde em **pid == 0** (Processo Filho) é realizada a execução do respetivo comando com **.executer()**. Em **pid > 0**, o Processo Pai espera pelo final de execução do Processo Filho com **.wait()** e, ao mesmo tempo, calcula o tempo de execução do comando. Por fim, é efetuada a construção da resposta através da *struct* “**Programa**” que, por sua vez, é enviada ao Controller.

3.2.2 .dispatch()

A função **.dispatch()** está encarregue, inicialmente, de criar o FIFO do Cliente: “**temp/comm_**” com **.mkfifo()**. Depois é criado através da *struct* “**Programa**”, a *request* para o Servidor com “**request.notification_type = 0**”. A *response* é criada do mesmo modo. Para garantir a robustez do sistema, esta função utiliza um ciclo **while** que avalia a resposta do Controller: se o servidor responder com -1 (*Full Queue*), o Runner adormece temporariamente e tenta submeter o pedido de novo, automaticamente, sem intervenção do utilizador. Assim que recebe permissão ou lugar na fila de espera, sai do ciclo, apaga o FIFO com **.unlink()** e devolve o controlo à **.main()**.

3.2.3 .parser()

A função **.parser()** recebe a string de comandos com ou sem *pipelines*, e encarrega-se de os dividir em tokens. É usada a função em C **.strtok()** para, quando é encontrado um comando de redirecionamento (i.e. “<”, “>”, “2>”), a execução seja compreendida pela **.main()** do cliente.

3.2.4 .cutpipe()

A **.cutpipe()** tem um comportamento semelhante à função anterior, mas esta apenas “tokeniza” quando encontra uma *pipeline* (“|”). Percorre a string de comandos até chegar ao último elemento. visto que é utilizada a função para execução: **.execvp()**, o último comando tem de ser **NULL**.

3.2.5 .executor()

A função **.executor()** começa, primeiramente, por percorrer o número de comandos interpretados pelo **.cutpipe()** abrindo os descritores para redirecionamento. É feito um **pid = fork()**. Em **pid == 0** é feito o redirecionamento de *std ins-outs* juntamente com a chamada da função **.parser()** e **.execvp()**. Em **pid > 0**, é feito o fechamento dos respectivos descritores.

3.3. Servidor (Controller.c)

3.3.1 .main()

A criação primordial da diretoria “**temp**” por parte da **.main()**, permite que os FIFO’s sejam colocados nessa diretoria temporária, facilitando a logística do Servidor. Só após a criação dessa mesma diretoria, é que pode ser criado o FIFO principal do programa “**temp/Communicator**”.

Além disso, são introduzidas duas *pipelines*: **int pipe1[2]** e **int pipe2[2]**. Essas *pipelines* permitem a comunicação entre duas funções extremamente essenciais à comunicação Comando-Queue (**.process()** e **.manager()**). Após a inicialização do Servidor com “\$./bin/Controller”, é escrito no ecrã do Servidor uma mensagem com a identificação dos “*write-read ends*” das *pipelines*, servindo de uma espécie de comprovativo que o Servidor iniciou com sucesso. Seguidamente, é feito um **pid1 = fork()** e **pid2 = fork()**. Em **pid1 == 0**, fechamos primeiramente os *ends* de leitura e escrita da primeira *pipeline* (**close(pipe1[0]); close(pipe2[1])**), chamando a função **.process()**. Em **pid2 == 0** fechamos, desta vez, os *ends* da segunda *pipeline* (**close(pipe2[0]); close(pipe2[1])**), chamando a função **.manager()**. Por fim fechamos ambos os *ends* das *pipelines* e esperamos por ambos os os processos filhos acabarem a execução com, nos dois casos, **wait(NULL)**.

3.3.2. **.process()**

A função **.process()** encarrega-se de, primeiramente, de abrir o “**temp/Communicator**” para receber o pedido de execução de um Cliente. É construído, através da *struct* “**Programa**” o comando a ser enviado para a *Queue* (através do **.manager()**) chamado de “**toSend_command**”. Se “**toSend_command.notification_type == 4**”, é reconhecido pelo **.process()**, que a intenção do Cliente é encerrar a execução do próprio Servidor, retornando a mensagem de encerramento ao Cliente, sem escrever para o **.manager()**. Se for outro tipo de “**notification_type**”, o **.process()** é obrigado a comunicar com o **.manager()** via *pipeline1* para conseguir enviar o comando para a gestão da *Queue*.

Após avaliação do pedido por parte das funções gestoras da *Queue*, o **.process()** recebe, através da *pipeline2*, a resposta ao pedido de execução, retornando a mensagem consoante a permissão, ou não, da execução do comando. Enviando assim a resposta para o Cliente via “**tem/comm_**”.

3.3.3. **.manager()**

A função **.manager()** responsabiliza-se por receber os pedidos enviados pelo **.process()** via *pipeline1*, sendo capaz de os gerir na *Queue*. A gestão da fila de espera é feita por duas funções auxiliares da **.manager()**: a **.enqueue()** e a **.dequeue()**. Após ser notificada pela **.dequeue()** que há comandos que podem ser executados, a função Manager envia uma resposta (obviamente junto com o comando) ao **.process()** via *pipeline2*.

3.3.4. **.enqueue()**

A função **.enqueue()** constitui o pilar de admissão e controlo do servidor, sendo responsável por validar a integridade dos limites do sistema antes de qualquer inserção de dados. O processo inicia-se com a verificação da capacidade disponível face à constante **MAX_QUEUE**, prevenindo eventuais *buffer overflows*. Havendo espaço, a tarefa é registada num *array*, utilizando o indicador/apontador de tamanho atual da fila como índice direto para a inserção. Por último, a

função gere o paralelismo de tarefas com a definição **MAX_PARALLEL**, determinando de forma dinâmica e efetiva se a tarefa transita para o estado de permissão de execução ou se permanece em estado de espera na *Queue*, notificando sempre o Controller do estado do pedido.

3.3.5 .dequeue()

Por fim, a função **.dequeue()** é responsável por, quando um processo é terminado de executar por parte do Cliente, removê-lo da *Queue*. Ou seja, a função **.dequeue()** apenas é chamada quando um cliente notifica ao servidor, o fim de execução de um comando, acedendo à *Queue* procurando tarefas com “**.running == 1**” (*flag* de final de execução de uma tarefa, definida em **lib.h**). Essa tarefa é marcada como “**found_index**”, e posteriormente, para **i = found_index**, é feita a desmarcação da tarefa/(s) atual/atuais na *Queue* com “**queue -> tasks[i] = queue -> tasks[i + 1]**”. Diminuindo os processos ativos, e o tamanho da fila de espera.

As tarefas com “**.running == 0**”, são marcadas como processos ativos, sendo o seu estado notificado ao Controller.

4. Considerações Finais

Em suma, o programa é capaz de representar uma simples comunicação Cliente/(s)-Servidor, retratando muito superficialmente o funcionamento de um Sistema Operativo. Foram implementadas em grande parte, funcionalidades vitais para o seu funcionamento.

É de notar que, funcionalidades mais avançadas como a implementação de *dups* não foram inseridas no projeto, o *standard output* da execução de comandos é diretamente escrito no terminal do Cliente.

Este projeto permitiu-nos consolidar o que fomos aprendendo com o decorrer deste semestre na UC de Sistemas Operativos.

Note-se que na função .dequeue() foi chamada, de uma maneira um pouco peculiar, a struct com “dummySet”. Esta foi uma forma que nós encontramos de o servidor não “crashar” com uma alta demanda de pedidos vinda de vários Clientes, mantendo sempre um end da pipeline aberto, aquando do envio de pedidos.

